Основы тестирования серверных приложений

Простые объяснения, примеры на Node.js и Internet Shop

Зачем изучать тестирование

- После изменения программы важно проверить, что ранее работавшие функции не сломались.
- Научимся писать проверки, которые можно запускать повторно.
- Будем использовать встроенные средства Node.js — устанавливать Jest пока не требуется.
- Новые термины сначала объясняем, затем применяем в коде.

Учебные вопросы

№	Тема	Что узнаем
1	Назначение тестирования	Зачем тесты и что такое регрессия
2	Уровни тестирования	Модульные, интеграционные, сквозные проверки
3	Как устроен тест	Сценарий, AAA, ожидаемый результат
4	Качество тестов и замены зависимостей	Изоляция, стабильность, Stub, Mock, Fake
5	Первый тест в Node.js	node:test, assert, запуск и ошибки

Учебный вопрос 1. Зачем тестировать приложение

- Тест — проверка поведения программы по заранее заданному ожидаемому результату.

Что такое тест

- Тест запускает код с определёнными данными и проверяет результат.
- Пример: цена 10 000 × 2 и цена 5 000 × 1 должны дать итог 25 000.
- Если результат отличается — тест сообщает об ошибке.
- Тест не доказывает, что ошибок нет вообще, но помогает обнаружить известные проблемы.

Ручная и автоматическая проверка

- Ручная: человек отправляет запрос и оценивает ответ — например, через Postman.
- Автоматическая: проверка записана в виде кода и выполняется командой.
- Повторный запуск помогает быстро проверять приложение после изменений.
- Ручная проверка тоже полезна для исследования интерфейса и необычных ситуаций.

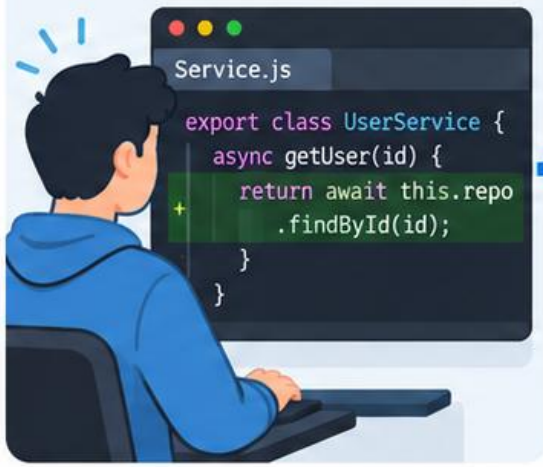
Регрессия — что это?

- Регрессия — новая ошибка в поведении, которое раньше работало.
- Пример: изменили расчёт скидки и сломали вычисление общей суммы.
- Пример: добавили проверку JWT и случайно запретили публичный каталог.
- Автоматические тесты помогают заметить такие изменения.

Изменение кода → test suite → pass/fail → решение о следующем шаге

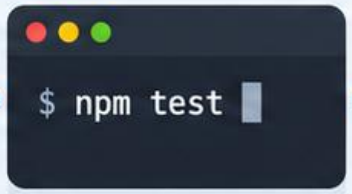
1. Разработчик изменяет код

Например, модуль Service



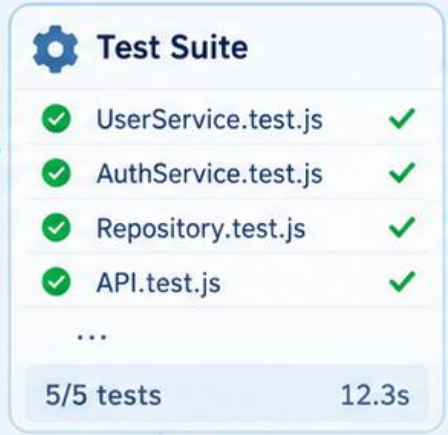
2. Запускает тесты

В терминале



3. Test suite выполняется

Запускаются все тесты проекта (юнит, интеграционные и др.)



PASS — Тесты прошли успешно



Tests passed
Все тесты успешно пройдены

Продолжить разработку
Можно добавлять новый функционал

Refactor
Безопасно улучшать код (рефакторинг)

FAIL — Тесты не прошли



Tests failed
Есть упавшие тесты

Найти причину
Изучить вывод тестов, логи, стек ошибок

Исправить код
Внести необходимые изменения в Service и др.

Повторный запуск
Снова запустить тесты (npm test)

Вернуться к test suite
Пока все тесты не пройдут

Тесты подсказывают, что делать дальше после изменения кода:
если **PASS** — можно двигаться дальше, если **FAIL** — нужно найти причину, исправить код и запустить тесты снова.

Что проверять в Internet Shop

- Расчёт стоимости заказа и скидки.
- Проверку цены и количества товара.
- Доступ к API: гость, пользователь, администратор.
- Создание товара и взаимодействие с Prisma — на следующем уровне тестирования.

Учебный вопрос 2. Уровни тестирования

- Не всякая проверка запускает весь Express и PostgreSQL.

Модульное тестирование (Unit)

- Проверяет отдельную функцию или небольшой модуль.
- Пример: calculateOrderTotal(items) правильно вычисляет сумму.
- Обычно не требует сервера Express и реальной базы данных.
- Такие проверки быстрые, а причину ошибки проще найти.

Функция для модульной проверки

```javascript
export function calculateOrderTotal(items) {
  return items.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0
  );
}
```

Что важно

- Функция получает массив товаров и суммирует стоимость.
- Мы можем проверить её, даже если Express и PostgreSQL выключены.

Интеграционное тестирование (Integration)

- Проверяет совместную работу нескольких компонентов.
- Пример: маршрут Express → контроллер → сервис → Prisma → тестовая база.
- Может обнаружить ошибки в обмене данными между слоями приложения.
- Обычно требует больше времени, чем модульный тест.

Сквозное тестирование (End-to-End, E2E)

- Проверяет законченный сценарий, близкий к реальному использованию.
- Пример: выполнить вход, создать заказ, затем получить его через API.
- Задействует многие компоненты, поэтому работает медленнее.
- Если проверка не прошла, причину нужно искать в нескольких слоях.

Пирамида тестирования

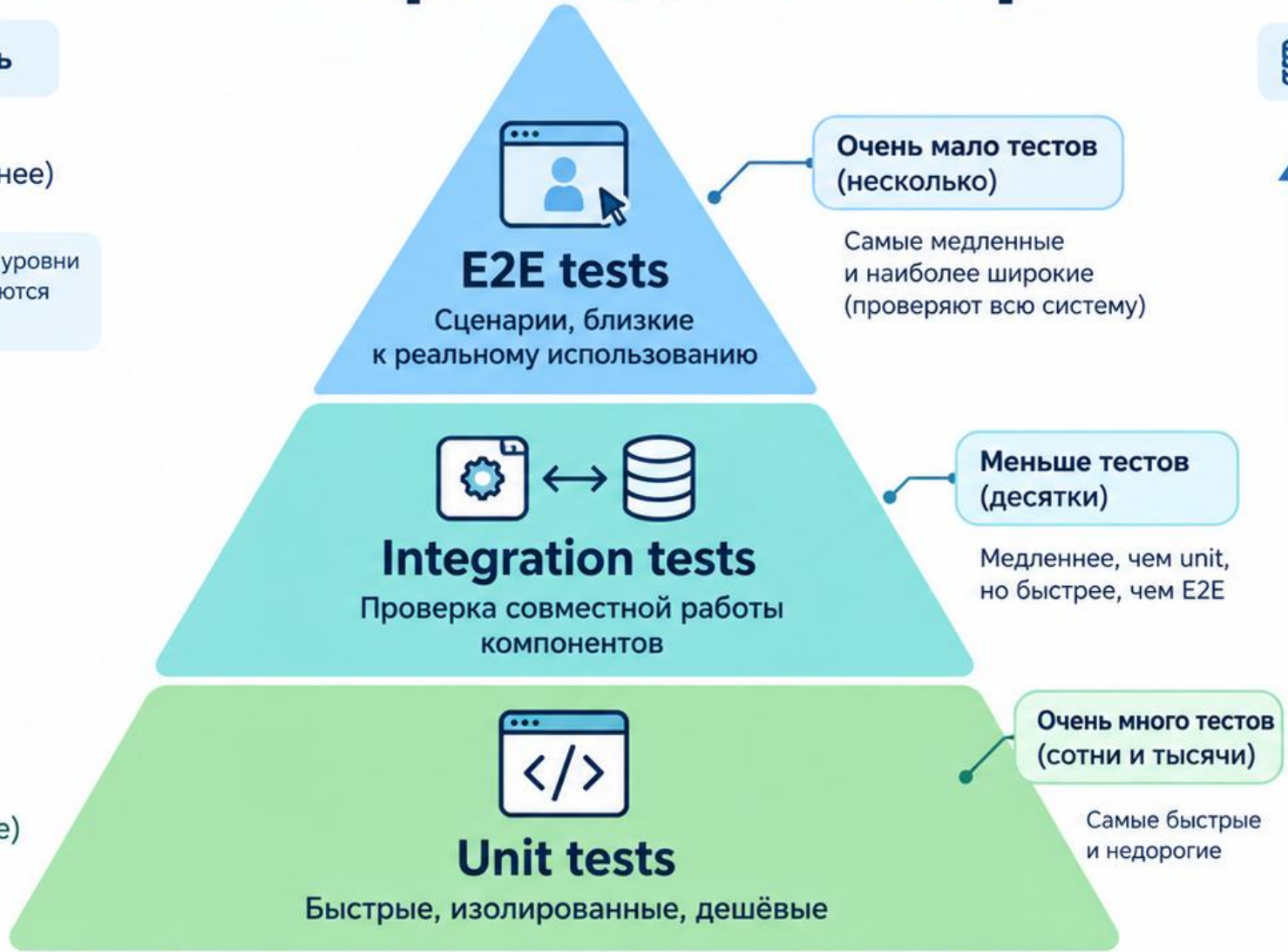
Скорость

Ниже
(медленнее)

Верхние уровни выполняются дольше

Выше
(быстрее)

Нижние уровни выполняются очень быстро



Очень мало тестов
(несколько)

Самые медленные и наиболее широкие (проверяют всю систему)

Меньше тестов
(десятки)

Медленнее, чем unit, но быстрее, чем E2E

Очень много тестов
(сотни и тысячи)

Самые быстрые и недорогие

Стоимость

Выше
(дороже)

Верхние уровни требуют больше ресурсов (время, окружение, поддержка)

Ниже
(дешевле)

Unit tests самые дешёвые в запуске и поддержке

Ширина проверки системы

Широкая
(вся система, пользовательский сценарий)

Наверху проверяется вся система от и до

Узкая
(отдельная функция или модуль)

Внизу проверяются отдельные части в изоляции

Больше тестов внизу, меньше — наверху

Быстрые unit tests покрывают детали, а немного медленных E2E tests проверяют весь пользовательский сценарий.

Сравнение уровней тестирования

Уровень	Что проверяем	Пример
Модульный	Отдельная функция	Расчёт стоимости заказа
Интеграционный	Связь нескольких компонентов	Express + Prisma + тестовая БД
Сквозной	Целый сценарий	Вход → заказ → ответ API

Зачем сочетать разные уровни

- Модульные тесты быстро проверяют вычисления и правила.
- Интеграционные обнаруживают проблемы на стыке модулей.
- Сквозные показывают, работает ли основной сценарий целиком.
- В одном проекте нужны разные проверки, а не только самые сложные.

Учебный вопрос 3. Как устроен тест

- Пишем понятный сценарий: данные → действие → проверка.

Тестовый сценарий

- Тестовый сценарий — описание условий, действия и ожидаемого результата.
- Пример: сумма двух товаров с ценами 10 000 и 5 000 при количестве 2 и 1.
- Ожидаемый результат: 25 000.
- Название сценария должно описывать поведение, например «считает сумму двух товаров».

Три этапа: Arrange – Act – Assert

- Arrange (подготовка): создаём исходные данные и ожидаемое значение.
- Act (действие): запускаем проверяемую функцию.
- Assert (проверка): сравниваем реальный результат с ожидаемым.
- Эту последовательность сокращённо называют AAA.

AAA: Arrange → Act → Assert

Arrange
test data

Test data

price	quantity
10000	2
5000	1

Variables

- items
- expectedTotal

Input objects

```
const items = [
  { price: 10000, quantity: 2 },
  { price: 5000, quantity: 1 }
];
```

Act
вызов calculateOrderTotal()

```
calculateOrderTotal(items)
```

Assert
actual === expected

```
const total = calculateOrderTotal(items);

const expected = 25000;
```

total === 25000

Структура хорошего теста: сначала подготовка данных, затем действие, затем проверка результата.

AAA на примере расчёта заказа

```javascript
import test from "node:test";
import assert from "node:assert/strict";
import { calculateOrderTotal } from "./orderMath.js";

test("считает сумму двух товаров", () => {
  // 1. Подготовка
  const items = [
    { price: 10000, quantity: 2 },
    { price: 5000, quantity: 1 }
  ];

  // 2. Действие
  const total = calculateOrderTotal(items);

  // 3. Проверка
  assert.strictEqual(total, 25000);
});
```

Что важно

- Название теста объясняет, что именно проверяется.
- strictEqual сравнивает фактическую сумму с ожидаемой.
- Если сумма не равна 25 000, тест завершится ошибкой.

Что такое утверждение (Assertion)

- Утверждение — условие, которое должно выполняться в тесте.
- Пример: assert.strictEqual(total, 25000) — сумма строго равна 25 000.
- Если утверждение не выполняется, тест считается неуспешным.
- Библиотека node:assert/strict содержит готовые функции проверки.

Основные способы проверки

Метод	Что проверяет	Пример
strictEqual(a, b)	Строгое равенство	total равен 25000
deepStrictEqual(a, b)	Содержимое массивов и объектов	Список товаров совпал
ok(value)	Истинность условия	Количество товаров > 0
throws(fn)	Ожидаемое исключение	Некорректное количество
rejects(promise)	Отклонение Promise	Ошибка асинхронной функции

Положительные и отрицательные проверки

- Положительный сценарий: передаём допустимые данные и ожидаем правильный результат.
- Отрицательный сценарий: передаём ошибочные данные и проверяем обработку ошибки.
- Пример: количество товара 2 — допустимо, 0 — недопустимо.
- Обе проверки нужны, чтобы обнаруживать разные типы ошибок.

Проверка граничных значений

- Ошибка может появиться на нижней или верхней границе разрешённого диапазона.
- Если количество допускается от 1 до 100: проверяем 1 и 100.
- Дополнительно проверяем 0 и 101 — программа должна отклонить их.
- Для массива с минимум одним элементом проверяем и пустой массив.

Учебный вопрос 4. Качество тестов

- Хороший тест повторяется с одинаковым результатом и не зависит от соседних тестов.

Изоляция тестов

- Изоляция означает, что один тест не должен влиять на другой.
- Каждый тест самостоятельно подготавливает необходимые данные.
- Запуск тестов в другом порядке не должен менять результат.
- Для интеграционных проверок используем отдельную тестовую базу данных.

Детерминированность тестов

- Детерминированный тест при одинаковых условиях даёт один и тот же результат.
- Проверка не должна случайно ломаться из-за времени, сети или общих данных.
- Если требуется текущее время, в тесте его лучше задавать явно.
- Результат теста должен зависеть от проверяемого поведения.

Нестабильный тест (Flaky test)

- Так называют тест, который то проходит, то не проходит без изменения проверяемого кода.
- Возможные причины: задержки сети, случайные данные и параллельное изменение общей БД.
- Такому тесту нельзя полностью доверять.
- Причину нестабильности нужно устранить, а не просто повторно запускать проверку.

Что такое зависимость программы

- Зависимость — внешний компонент, который использует проверяемый код.
- Пример: сервис товаров обращается к Prisma для получения данных из PostgreSQL.
- Другие примеры: отправка электронной почты, файловое хранилище, платёжное API.
- В модульном тесте часто полезно заменить зависимость контролируемым объектом.

Зачем нужны тестовые замены (Test doubles)

- Тестовая замена — объект или функция вместо настоящей зависимости.
- Пример: вместо реальной базы данных возвращаем заранее известный товар.
- Так мы проверяем сервис без сети, PostgreSQL и побочных действий.
- Разные виды замен называются Stub, Mock и Fake.

Stub, Mock и Fake: главное различие

Вид	Определение	Пример
Stub (заглушка)	Возвращает заранее заданные данные	findProduct() всегда возвращает один товар
Mock (имитация с проверкой вызовов)	Позволяет проверить, был ли вызов и с какими аргументами	Проверяем, вызвана ли отправка письма
Fake (упрощённая реализация)	Реально выполняет простые операции вместо сложного компонента	Хранит товары в обычном массиве

Stub — заглушка с заранее известным ответом

```javascript
const productRepository = {
  async findById(id) {
    return { id: 1, name: "Ноутбук" };
  }
};

const product = await productRepository.findById(1);
console.log(product.name); // Ноутбук
```

Что важно

- Здесь нет соединения с PostgreSQL.
- На любой вызов метод выдаёт подготовленный объект.
- Это полезно, когда нас интересует поведение сервиса, а не работа базы.

Mock — проверяем вызов зависимости

```javascript
let sentTo = null;
const mailService = {
  async send(to, message) {
    sentTo = to;
  }
};

await mailService.send("user@example.com", "Заказ при
assert.strictEqual(sentTo, "user@example.com");
```

Что важно

- Имитация фиксирует, кому попытались отправить письмо.
- Проверяем факт вызова и аргумент — настоящее письмо не отправляется.
- В реальных проектах такую проверку часто автоматизируют средствами библиотеки тестирования.

Fake — упрощённое хранилище товаров

```javascript
const products = [];
const fakeRepository = {
  add(product) {
    products.push(product);
  },
  findAll() {
    return products;
  }
};

fakeRepository.add({ id: 1, name: "Ноутбук" });
assert.strictEqual(fakeRepository.findAll().length,
```

Что важно

- Упрощённая реализация действительно сохраняет товар, но только в массиве.
- Она работает по смыслу похоже на хранилище, но не использует PostgreSQL.
- Данные исчезнут после завершения процесса.

Какую замену выбрать

- Stub — когда нужны заранее подготовленные результаты от зависимости.
- Mock — когда важно проверить взаимодействие: вызвали метод или нет, что передали.
- Fake — когда нужна простая работающая реализация вместо настоящей системы.
- На этой лекции достаточно понимать разницу; специализированные библиотеки рассмотрим позже.

Отдельное окружение для тестов

- Тесты не должны использовать рабочую базу данных приложения.
- Рабочая (production) база может содержать настоящие заказы и пользователей.
- Для интеграционных тестов задавайте отдельную DATABASE_URL и тестовую БД.
- Для простых модульных тестов база данных обычно вообще не нужна.

Учебный вопрос 5. Первый автоматический тест Node.js

- Используем встроенные модули node:test и node:assert/strict.

Что такое node:test

- node:test — встроенный модуль Node.js для объявления и выполнения тестов.
- Отдельно устанавливать его через npm не требуется.
- Функция test("название", () => ...) задаёт тестовый сценарий.
- Модуль node:assert/strict проверяет ожидаемый результат.

Какие файлы создадим

- services/orderMath.js — функция вычисления суммы.
- tests/orderMath.test.js — файл с проверками этой функции.
- Имена и пути должны совпадать с импортами в примерах.
- Для ES-модулей в package.json используется "type": "module".

Файл services/orderMath.js

```javascript
export function calculateOrderTotal(items) {
  if (!Array.isArray(items)) {
    throw new Error("items должен быть массивом");
  }
  return items.reduce((sum, item) => {
    if (!Number.isInteger(item.quantity) || item.quan
      throw new Error("quantity должен быть больше ну
    }
    return sum + item.price * item.quantity;
  }, 0);
}
```

Что важно

- Мы добавили проверку количества, чтобы отрицательный тест был действительно выполнимым.
- Если quantity равен 0, функция выбрасывает исключение.
- Для учебного примера проверяются только основные условия; полноценная валидация может быть шире.

Файл tests/orderMath.test.js

```javascript
import test from "node:test";
import assert from "node:assert/strict";
import { calculateOrderTotal } from "../services/orde

test("сумма товаров равна 25000", () => {
  const items = [
    { price: 10000, quantity: 2 },
    { price: 5000, quantity: 1 }
  ];
  assert.strictEqual(calculateOrderTotal(items), 2500
});
```

Что важно

- Импорт берёт функцию из соседней папки services.
- Ожидаемый ответ вычисляется заранее: 10 000 × 2 + 5 000 = 25 000.
- Node.js сообщит об ошибке, если фактическое значение отличается.

Проверка ошибочного количества

```javascript
test("отклоняет количество 0", () => {
  const items = [{ price: 10000, quantity: 0 }];

  assert.throws(
    () => calculateOrderTotal(items),
    /quantity/
  );
});
```

Что важно

- assert.throws() ожидает, что вызов функции завершится исключением.
- Регулярное выражение /quantity/ проверяет часть текста ошибки.
- Проверка сработает, поскольку наша функция действительно выбрасывает ошибку при количестве 0.

Запуск тестов

```bash
node --test
```

Что важно

- Команду выполняем из корня Node.js-проекта.
- Node.js находит подходящие тестовые файлы и запускает проверки.
- Если тест не проходит, в терминале выводятся сведения об ошибке.

Как читать результаты

- Успешный тест: фактический результат совпал с ожидаемым.
- Неуспешный тест: проверка не выполнилась или произошла неожиданная ошибка.
- Нужно посмотреть название теста, сообщение и место ошибки.
- После исправления кода запускаем проверки повторно.

Что делать после изменения кода

- Изменили функцию или обработчик API.
- Запустили автоматические тесты.
- При ошибке — изучили сообщение и исправили причину.
- Когда тесты проходят — можно продолжать разработку, сохраняя осторожность: проверки не гарантируют отсутствие всех ошибок.

Мини-практика: проверка функции заказа

Действие	Ожидаемый результат
Создать orderMath.js	Функция экспортируется
Создать orderMath.test.js	Есть положительный и отрицательный тест
Выполнить node --test	Оба теста проходят
Заменить ожидаемую сумму на 24000	Первый тест показывает ошибку
Вернуть 25000	Снова оба теста проходят

Контрольные вопросы

1. Что такое автоматический тест и регрессия?
2. Чем отличаются модульные, интеграционные и сквозные тесты?
3. Из каких этапов состоит метод AAA?
4. Что такое утверждение и для чего нужен strictEqual()?
5. Зачем проверять граничные значения?
6. Что такое изоляция и детерминированность теста?
7. Чем различаются Stub, Mock и Fake?
8. Почему тесты не должны работать с рабочей базой?
9. Для чего нужны node:test и node:assert/strict?
10. Как запустить тесты и понять причину неудачи?

Итоги занятия

1. Тесты сравнивают фактический результат с ожидаемым и помогают обнаруживать регрессии.
2. Модульные проверки быстры; интеграционные проверяют связи компонентов; сквозные — целые сценарии.
3. Структура AAA разделяет подготовку, действие и проверку.
4. Нужны положительные, отрицательные и граничные сценарии.
5. Хорошие тесты независимы друг от друга и воспроизводимы.
6. Stub возвращает данные, Mock проверяет взаимодействие, Fake упрощённо реализует зависимость.
7. Для первых тестов достаточно встроенных node:test и node:assert/strict.
8. Тестировать нужно на искусственных данных, не затрагивая рабочую базу.